ML2++: Dual-Mode Modeling and Automated Execution of IoT Time-Series Forecasting Pipelines

Zahra Mardani Korani
LNEC & ISCTE-IUL / ISTAR
Lisbon, Portugal
zmardani@lnec.pt

Thimoty Smet
University of Antwerp & Flanders Make
Antwerp, Belgium
thimoty.smet@student.uantwerpen.be

Moharram Challenger
University of Antwerp & Flanders Make
Antwerp, Belgium
moharram.challenger@uantwerpen.be

Armin Moin
University of Colorado Colorado Springs
Colorado Springs, CO, USA
amoin@uccs.edu

João Carlos Ferreira
ISCTE-IUL / ISTAR & Molde University College
Lisbon, Portugal
joam@himolde.no

Alberto Rodrigues da Silva
INESC-ID & Instituto Superior Técnico, Universidade de Lisboa
Lisbon, Portugal
alberto.silva@tecnico.ulisboa.pt

Gonçalo Vitorino Jesus
LNEC
Lisbon, Portugal
gjesus@lnec.pt

Abstract

Integrating data analytics, machine learning (ML), and time-series forecasting into IoT systems often requires engineers to implement and maintain preprocessing, training, evaluation, and visualization pipelines outside the system design models. ML2++ addresses this gap by extending ThingML with analytics-aware modeling constructs that let users declare data sources, preprocessing steps, forecasting/ML algorithms, evaluation metrics, and visualization settings directly at the modeling level—without manual scripting. We present ML2++ as a dual-editor framework that supports both textual modeling in Textula and graphical modeling in Sirius-Web. The two editors share a common metamodel and consistent language concepts, enabling users to specify complete predictive IoT workflows in their preferred modality. Both environments generate executable Python/Java projects; Textula emphasizes fast textual authoring and supports local project generation and execution (or download for offline runs), while Sirius-Web is tightly integrated with a web-based Orchestrator that automates project generation and server-side execution and returns interpretable outputs, including logs, evaluation metrics, and visualization plots. Through an end-to-end tool demonstration on representative IoT forecasting scenarios, we show how ML2++ improves productivity, reproducibility, and maintainability by unifying system design and data-driven pipeline specification within a single model-driven workflow.

CCS Concepts

• **Software and its engineering** → *Software prototyping*; **Domain specific languages**; • **Mathematics of computing** → *Machine learning*.

Keywords

model-driven engineering, domain-specific modeling, IoT, time-series forecasting, code generation

1 Introduction

Machine learning (ML) and time-series forecasting have become core capabilities of modern Internet of Things (IoT) applications, supporting predictive maintenance, environmental monitoring, and decision support [2]. However, in practice, developing analytics-enabled IoT systems still requires substantial manual effort. Engineers typically implement preprocessing, feature engineering, model training, evaluation, prediction, and visualization pipelines outside of system design models, using heterogeneous scripts and tools. This separation complicates reproducibility, increases maintenance costs, and makes iterative changes such as modifying a forecast horizon or adding an input feature prone to errors and time-consuming [1, 3, 4, 8, 9].

Model-Driven Engineering (MDE) addresses these challenges by raising the level of abstraction and enabling the automatic generation of executable artifacts from models [8]. Domain-Specific Modeling Languages (DSMLs) have been successfully applied to capture the structure and behavior of the IoT system, yet analytics workflows are often delegated to external ML tools and libraries [1, 4, 5]. The ML2 family of DSMLs (ML2, ML2+, and ML2++) bridges this gap by integrating ML and time-series forecasting constructs directly into IoT modeling [6, 7, 10, 11]. In particular, ML2++ extends prior work with support for multi-step forecasting, automated visualization, multiple algorithm families, and a dual-mode modeling experience that accommodates different user preferences and workflows [7].

This paper focuses on demonstrating ML2++ as a practical tool rather than providing a detailed metamodel or architectural analysis. ML2++ is presented as a dual-editor environment that allows users to specify complete IoT forecasting pipelines either textually, using the Textula editor, or graphically, using the Sirius-Web editor [7]. Both editors rely on a shared metamodel and generate consistent executable Python and Java projects that implement preprocessing, training, forecasting, evaluation, and visualization [7].

While Textula emphasizes fast textual authoring and supports local or offline execution of generated projects, Sirius-Web is tightly integrated with a web-based Orchestrator that automates project generation and server-side execution and provides immediate access to execution logs, evaluation metrics, and diagnostic plots [7].

The tool demonstration follows an end-to-end workflow based on a representative IoT time-series forecasting scenario. Starting from a high-level specification of system structure and analytics configuration, users select datasets, define preprocessing steps, configure forecast horizons and algorithms, and generate runnable artifacts that can be executed to inspect results [6, 7]. The demonstration highlights how ML2++ enables a compact model–generate–run–inspect cycle and supports rapid experimentation while preserving traceability and reproducibility [7, 8, 10].

The remainder of the paper is organized as follows. Section 2 introduces the ML2++ tool and its architecture. Section 2.2 presents the textual modeling workflow with Textula, while Section 2.3 demonstrates the graphical workflow using Sirius-Web and Orchestrator. Section 3 discusses related work and positions ML2++ with respect to existing model-driven and analytics-oriented approaches. Finally, Section 5 concludes the paper and outlines directions for future work.

2 ML2++ (Tool)

ML2++ is a dual-mode, model-driven environment for specifying and executing end-to-end IoT time-series forecasting pipelines. Users model both (i) the IoT system structure and communication, and (ii) the associated analytics pipeline (data selection, preprocessing, learning/forecasting configuration, evaluation, and visualization) within a single ML2++ model. From this model, ML2++ generates an executable project that implements the configured pipeline and produces standardized runtime outputs such as logs, evaluation metrics, and plots [5, 7].

2.1 Architecture

Figure 1 summarizes the ML2++ workflow from modeling to execution and outputs.

ML2++ provides two complementary front-ends for authoring the same underlying model. Textula offers a browser-based textual editor for rapid DSL authoring with validation and one-click actions. Sirius-Web provides a browser-based graphical editor that combines a modeling canvas, a model explorer, and form-based configuration pages for the analytics pipeline. Both editors persist models against a shared metamodel, ensuring that equivalent pipeline concepts (data, preprocessing, algorithm configuration, evaluation, and visualization) can be expressed consistently in either modality [7].

Once a model is defined, a code-generation layer compiles it into a runnable Python/Java project that implements the complete pipeline. In the current prototype, generation includes (i) preprocessing scripts for data loading, alignment/resampling, missing-value handling, and scaling/normalization; (ii) training scripts for the selected algorithm family; (iii) multi-step forecasting scripts for the configured lag and forecast horizon; and (iv) visualization

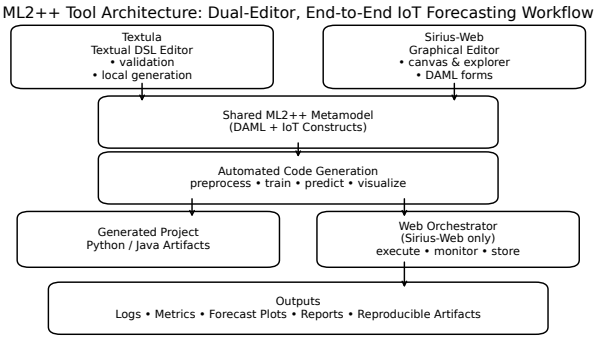

Figure 1: ML2++ workflow: models authored textually (Textula) or graphically (Sirius-Web) share a metamodel, are compiled into runnable projects, and are executed locally or via the Orchestrator to produce logs, metrics, plots, and downloadable artifacts.

scripts that generate diagnostic plots such as preprocessing previews, learning curves, and forecast-versus-actual views [7]. Generated projects can be executed locally (e.g., after download), or executed through the web-based Orchestrator.

The Orchestrator automates project generation and server-side execution and organizes all produced artifacts in a structured workspace. It returns interpretable results through a web interface, including execution logs, evaluation metrics, and a plot gallery. This supports an iterative model–generate–run–inspect cycle, where changes to the pipeline configuration are applied at the model level and propagated consistently by regeneration and re-execution [7].

2.2 Textual Editor (Textula)

Textula[1] is the browser-based textual editor of ML2++. It supports fast authoring of ML2++ models with syntax highlighting and validation, enabling users to specify (i) the ThingML-like IoT structure (messages, ports, things, and configurations) and (ii) the analytics pipeline, expressed as a declarative `data_analytics` block. The resulting model is compiled into an executable project that contains the scripts required for preprocessing, training/loading, multi-step forecasting, evaluation, and visualization [5, 7].

What users specify in Textula. A Textula model combines system-level communication with analytics configuration in a single artifact. The IoT portion defines the components that exchange data (e.g., sensor observations and prediction results) and how they are connected at runtime. The analytics portion declares the dataset and selected features, preprocessing directives, time-series framing parameters (lag and forecast horizon), algorithm configuration, evaluation settings, and requested plots. Capturing these elements at the model level keeps pipeline configuration explicit and versionable, while allowing regeneration after each change [7].

Workflow. Textula supports a lightweight model–generate–run–inspect cycle. Users validate and save the model, optionally preview

[1] https://ml2plusplus.jvmhost.net/login.html

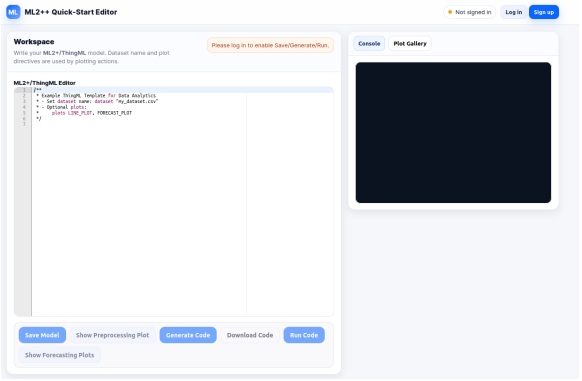

Figure 2: Textula: textual authoring of an ML2++ model that combines IoT communication structure with a `data_analytics` pipeline specification.

preprocessing to check data quality, and generate a runnable project. The generated project can be executed in the provided environment or downloaded for offline execution, while the produced artifacts (logs and plots) allow users to inspect pipeline behavior and outcomes [7].

Minimal `data_analytics` *example.* Listing 1 shows a minimal `data_analytics` specification for multi-step forecasting. The block declares the dataset and features, selects preprocessing options, configures a time-series framing (lag and forecast steps), chooses an algorithm, and requests a forecast-versus-actual plot. From such a concise model-level specification, ML2++ generates executable scripts for the full pipeline.

Listing 1: Minimal ML2++ `data_analytics` specification for multi-step forecasting.

```
data_analytics flowTS @dalib "keras-tf" {
  data { dataset "flow.csv"; features level temp; output_features
       flow; }
  preprocessing { fill_missing INTERPOLATION; scaler Standard; }
  time_series { lag 5; steps 3; supervised TRUE; }
  model { algorithm LSTM(neurons=32, epochs=50); }
  visualization { plots FORECAST_VS_ACTUAL; }
}
```

2.3 Graphical Modeling with Sirius-Web and the Orchestrator

ML2++ also supports graphical instance modeling through a Sirius-Web front-end that is integrated with a web-based execution service called the **Orchestrator**. Sirius-Web provides a canvas-based editor and form-driven configuration views that guide users through constructing an ML2++ model without writing DSL syntax. Models authored graphically are persisted against the same metamodel as Textula models; therefore, the graphical and textual workflows are semantically equivalent and produce consistent generated Python/-Java artifacts [7].

Environment setup. Sirius-Web and the Orchestrator are distributed as an integrated web environment and can be launched locally using Docker. After startup, users access Sirius-Web in the browser to create or import an ML2++ project and edit the model

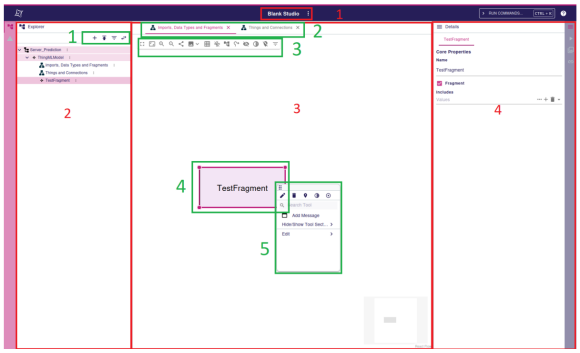

Figure 3: Sirius-Web: graphical instance modeling of an ML2++ application using the model explorer, canvas, and property views.

graphically. The Orchestrator provides execution controls and a results view for inspecting logs, evaluation metrics, and generated plots [7].

Graphical model construction. In Sirius-Web, users build an ML2++ instance model that captures the IoT application structure (e.g., sensors, analytics components, and their connections). The editor combines a model explorer for navigating the instance hierarchy, a graphical canvas for visualizing elements and links, and a properties/form view for editing the attributes of the selected element. This supports incremental modeling, where users can introduce components and refine their configuration while immediately seeing the system structure on the canvas.

Configuring the analytics pipeline. Analytics configuration is performed through dedicated multi-page forms that correspond to the structure of the `data_analytics` specification. Selecting a `data_analytics` element opens pages for dataset and feature selection, preprocessing directives, time-series parameters (lag and forecast horizon), algorithm configuration, evaluation metrics, and visualization settings. By exposing algorithm-specific hyperparameters through forms, Sirius-Web reduces syntactic overhead and supports guided configuration while preserving the same model semantics as the textual workflow [7].

Execution and result inspection. After modeling and configuration, users trigger project generation and execution through the Orchestrator. The Orchestrator compiles the model into a runnable project, executes preprocessing, training (or model loading), and multi-step forecasting, and collects the produced artifacts. Results are presented through a web interface, including streamed execution logs, reported evaluation metrics, and a plot gallery with diagnostic visualizations such as preprocessing summaries, training curves, and forecast-versus-actual views. This enables users to iteratively refine the model and re-run the pipeline while keeping configuration changes traceable at the model level [7].

3 Availability and Replication

We provide a public artifact for replicating the ML2++ tool demonstration, including source code, example models/datasets, and deployment scripts for both front-ends. The repository contains the

Figure 4: Sirius-Web: form-based configuration of an ML2++ `data_analytics` pipeline, including time-series parameters, algorithm settings, and visualization options.

Eclipse/Xtext source of Textula (and demo models), and a web deployment is available at https://ml2plusplus.jvmhost.net/. It also provides the source code and a Docker setup to run Sirius-Web together with the Orchestrator, enabling users to import an example project, configure the `data_analytics` pipeline, execute the workflow server-side, and inspect logs, metrics, and plots in the UI. A short demonstration video is available at https://youtu.be/B_KAssp8E5E. All materials are available at https://github.com/micss-lab/ML-QuadratPP.

4 Related Work

Model-Driven Engineering (MDE) has been widely explored as a means to raise the abstraction level of software development and to generate executable artifacts from models, including for IoT systems and cyber-physical applications [5, 8]. In parallel, machine learning practice typically relies on code-centric frameworks and notebook-based workflows (e.g., TensorFlow and Keras), which provide algorithmic flexibility but often require substantial manual effort to orchestrate preprocessing, training, evaluation, and visualization across scripts and tools [1, 4]. Visual and low-code analytics platforms such as KNIME and RapidMiner reduce the boilerplate for pipeline assembly, yet remain detached from system-level IoT models and do not provide a unified modeling abstraction for IoT structure and analytics behavior [3, 9].

Within the MDE-for-ML space, previous work has proposed DSMLs and model-based approaches to integrate ML into software engineering processes and to support the systematic development of ML-enabled systems [8, 10]. The ML-Quadrat line of work introduced ML2 and ML2+ as DSML-based approaches that embed ML and time-series forecasting concepts into IoT modeling, allowing the generation of code from model-level specifications [6, 11]. ML2++ extends this direction with dual-mode modeling (textual and graphical), automated visualization generation, and an execution-oriented Orchestrator integration that compiles and runs complete time-series pipelines while returning structured outputs (logs, metrics, and plots) for inspection and iteration [7].

Compared to code-centric development, ML2++ emphasizes traceability and reproducibility by representing the full analytics configuration as a versionable model and by consistently regenerating the pipeline after each change. Compared to low-code analytics tools, ML2++ maintains an explicit connection between the IoT system model and the analytics pipeline, allowing developers to evolve both perspectives within a single model-driven workflow [7].

5 Conclusion

This tool demonstration presented ML2++, a dual-mode modeling environment for building IoT time-series forecasting pipelines from model-level specifications. The demonstration showed how users can define the same analytics-enabled IoT workflow either textually in Textula or graphically in Sirius-Web, relying on a shared metamodel to preserve consistent semantics across both modalities. From the model, ML2++ generates an executable Python/Java project that implements preprocessing, training/loading, multi-step forecasting, evaluation, and visualization, enabling changes such as adjusting the forecast horizon or modifying selected features to be applied at the model level and propagated through regeneration.

We also demonstrated the Orchestrator-based execution workflow, where generated projects are executed server-side, and results are returned through a web interface. The outputs produced include execution logs, evaluation metrics, and a gallery of diagnostic plots that support iterative pipeline inspection and refinement. To facilitate reuse and replication, we provide the tool, sample models, and datasets, and a Docker-based deployment package as a public artifact.

Future work includes extending algorithm and visualization coverage, strengthening support for additional IoT deployment targets, and conducting larger-scale user studies to quantify modeling effort, usability, and productivity benefits across diverse user backgrounds.

References

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafał Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. White paper / arXiv preprint. arXiv:1603.04467 [cs.DC] https://arxiv.org/abs/1603.04467

[2] Darwin Alulema, Javier Criado, Luis Iribarne, Antonio Jesús Fernández-García, and Rosa Ayala. 2020. A model-driven engineering approach for the service integration of IoT systems. *Cluster Computing* 23, 3 (2020), 1937–1954.

[3] Michael R. Berthold, Nicolas Cebron, Fabian Dill, Thomas R. Gabriel, Tobias Kötter, Thorsten Meinl, Peter Ohl, Christoph Sieb, Kilian Thiel, and Bernd Wiswedel. 2009. KNIME: The Konstanz Information Miner: Version 2.0 and Beyond. *SIGKDD Explorations* 11, 1 (2009), 26–31. doi:10.1145/1656274.1656280

[4] François Chollet. 2015. Keras. GitHub repository / software. https://github.com/fchollet/keras

[5] Nicolas Harrand, Franck Fleurey, Brice Morin, and Knut Eilif Husa. 2016. ThingML: A Language and Code Generation Framework for Heterogeneous Targets. In *Proceedings of the 19th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS)*. doi:10.1145/2976767.2976812

[6] Zahra Mardani Korani, Moharram Challenger, Armin Moin, João Carlos Ferreira, Alberto Rodrigues da Silva, and Gonçalo Vitorino Jesus. 2025. From ML2 to ML2+: Integrating Time Series Forecasting in Model-Driven Engineering of Smart IoT Applications. In *Proceedings of the MBSE-AI Integration Workshop – 2nd Workshop on Model-Based Systems Engineering and Artificial Intelligence*.

[7] Zahra Mardani Korani, Moharram Challenger, Armin Moin, João Carlos Ferreira, Alberto Rodrigues da Silva, and Gonçalo Vitorino Jesus. 2025. ML2++: A Model-Driven Blended Framework for Automated Machine Learning, Time Series Forecasting, and Data Visualization in IoT Applications. In *Proceedings of the*

MoDIoT Workshop of the ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS 2025). Grand Rapids, Michigan, USA. doi:10.5281/zenodo.17989836 Publication date: 2025-10-05.

[8] Ana C. Marcén, Antonio Iglesias, Raúl Lapeña, Francisca Pérez, and Carlos Cetińa. 2024. A Systematic Literature Review of Model-Driven Engineering using Machine Learning. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3430514

[9] Ingo Mierswa, Michael Wurst, Ralf Klinkenberg, Martin Scholz, and Timm Euler. 2006. Yale (now: RapidMiner): Rapid Prototyping for Complex Data Mining Tasks. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 935–940.

[10] Armin Moin, Moharram Challenger, Atta Badii, and Stephan Günnemann. 2022. A model-driven approach to machine learning and software modeling for the IoT. *Software and Systems Modeling* 21, 3 (June 2022), 987–1014. doi:10.1007/s10270-021-00967-x

[11] Armin Moin, Andrei Mituca, Moharram Challenger, Atta Badii, and Stephan Günnemann. 2021. ML-Quadrat & DriotData: A Model-Driven Engineering Tool and a Low-Code Platform for Smart IoT Services. arXiv preprint. arXiv:2107.02692 [cs.SE] https://arxiv.org/abs/2107.02692